# Software Requirements Specification
for

# Hostel Management System

**Version <1.0>**

**Prepared by**

**Group Number: 42**


| | | |
|---|---|---|
| **NJ Madhav** | **SE23UCSE122** | **Se23ucse122@mahindrauniversity.edu.in** |
| **Tathya Sharma** | **SE23UCSE173** | **Se23ucse173@mahindrauniversity.edu.in** |
| **Krishna Sai** | **SE23UMCS064** | **Se23umcs064@mahindrauniversity.edu.in** |


**Instructor:** **Arun Avinash Chauhan**

**Course:** **Software Engineering**

**Lab Section:** ***ground floor lab, IT BLOCK 2***

**Teaching Assistant:** **Sai Srithaja**

**Date:** **13th March 2026**

## Software Requirements Specification

## Revisions

| Version | Primary Author(s) | Description | Date |
|---|---|---|---|
| 1.0 | Team 42 | Initial SRS Document | March 2026 |

# 1. Introduction

## 1.1 Document Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Hostel Management System developed by Team 42. The system aims to digitize hostel operations including room allocation and maintenance complaint management using a ticket-based approach.

This document is intended for the development team, the course instructor, and the evaluating professor. It covers the complete scope of the system and serves as the primary reference throughout the project lifecycle.

## 1.2 Product Scope

The Hostel Management System is a web-based application that allows students to submit maintenance complaints, request room allocations, and track ticket status in real time. Wardens manage tickets for their block while Admins have full system control.

Key benefits include reduced manual work, faster complaint resolution, better communication between students and management, and a transparent trackable record of all maintenance issues.

## 1.3 Intended Audience and Document Overview

This document is intended for the development team (Team 42), the course instructor, and testers. It is organized as follows: Section 1 provides an introduction, Section 2 gives an overall product description, Section 3 details specific requirements, and the appendices contain the data dictionary and group log.

## 1.4 Definitions, Acronyms and Abbreviations

| Term | Definition |
|---|---|
| HMS | Hostel Management System |
| SRS | Software Requirements Specification |
| JWT | JSON Web Token — used for secure authentication |
| API | Application Programming Interface |
| CRUD | Create, Read, Update, Delete — basic database operations |
| UI | User Interface |

## 1.5 Document Conventions

This document follows IEEE formatting requirements using Arial font size 11. Requirements are labeled F (Functional) or NF (Non-Functional) followed by a number.

## 1.6 References

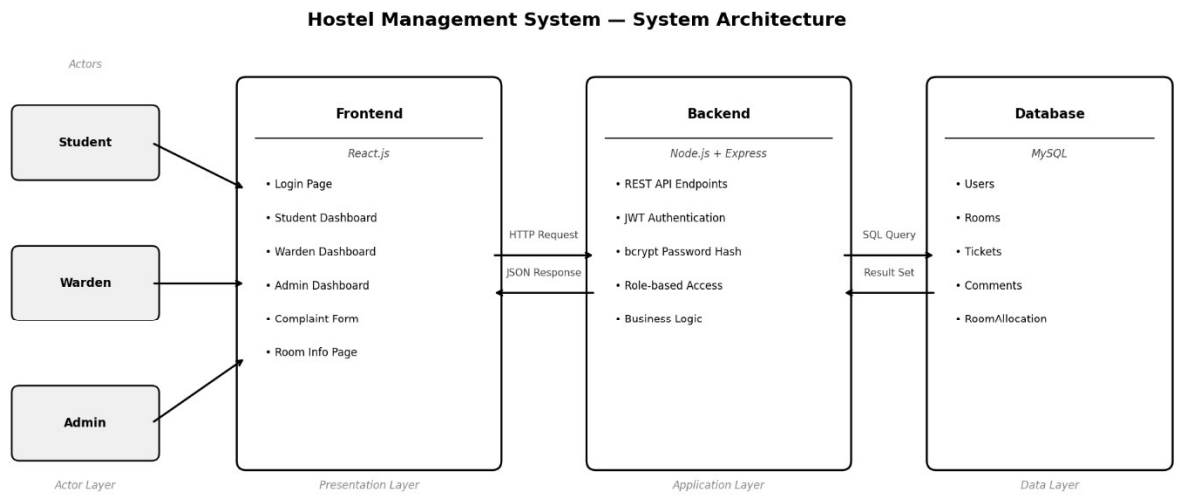

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- React Documentation — https://react.dev
- Node.js Documentation — https://nodejs.org
- MySQL Documentation — https://dev.mysql.com/doc

# 2. Overall Description

## 2.1 Product Overview

The Hostel Management System is a new self-contained web application built for managing hostel operations at a single institution. It replaces manual paper-based processes with a digital platform. The system has three components: a React frontend, a Node.js/Express backend, and a MySQL database.

**Figure 1: System Architecture Diagram**




| Component | Description |
|---|---|
| Frontend (React.js) | Web-based UI accessible via browser |
| Backend (Node.js) | REST API handling all business logic |
| Database (MySQL) | Stores users, rooms, tickets, comments |
| Auth (JWT + bcrypt) | Secure login and role-based access control |

## 2.2 Product Functionality

- User authentication with role-based access (Student, Warden, Admin)
- Room allocation management — assign and view room details
- Maintenance complaint submission with category and priority
- Ticket tracking with status updates (Pending, In Progress, Resolved)
- Comment/chat thread on each ticket between student and staff
- Admin dashboard with system-wide statistics and controls
- Warden dashboard for block-level ticket management

## 2.3 Design and Implementation Constraints

- Tech stack: React.js (frontend), Node.js + Express (backend), MySQL (database)
- Development must be completed by 11th May 2026
- System designed for a single hostel institution only
- Authentication must use JWT tokens, passwords hashed using bcrypt
- Team consists of 3 members with limited development time

### 2.4 Assumptions and Dependencies

- Users are pre-registered — students receive credentials from admin
- Stable internet connectivity is available for all users
- Complaint categories (Electrical, Furniture, Cleanliness) are fixed
- The hosting environment supports Node.js and MySQL

# 3. Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

- Login page with role selector (Student, Warden, Admin)
- Sidebar navigation specific to each role
- Dashboard pages with statistics and data tables
- Forms for submitting complaints and room change requests

### 3.1.2 Hardware Interfaces

- Desktop or laptop with a modern web browser
- Internet connectivity (wired or wireless)

### 3.1.3 Software Interfaces

- Frontend communicates with backend via REST API over HTTP
- Backend connects to MySQL using the mysql2 Node.js driver
- JWT tokens passed in HTTP Authorization headers

## 3.2 Functional Requirements

### F1 — User Authentication

- F1.1: The system shall allow users to log in using email and password
- F1.2: The system shall support three roles: Student, Warden, and Admin
- F1.3: The system shall generate a JWT token on successful login
- F1.4: The system shall restrict page access based on user role
- F1.5: The system shall allow users to log out

### F2 — Room Management

- F2.1: Admin shall be able to add, edit, and view rooms
- F2.2: Admin shall be able to assign students to rooms
- F2.3: Room status shall be displayed as Vacant, Partial, or Occupied
- F2.4: Students shall be able to view their assigned room details
- F2.5: Students shall be able to submit a room change request

### F3 — Complaint / Ticket System

- F3.1: Students shall be able to submit a maintenance complaint
- F3.2: Each complaint shall include title, description, category, and priority
- F3.3: Complaint categories shall be: Electrical, Furniture, Cleanliness
- F3.4: Each ticket shall be assigned a unique ID and timestamp automatically
- F3.5: Ticket status shall follow: Pending → In Progress → Resolved
- F3.6: Students shall be able to view all their submitted tickets
- F3.7: Wardens shall be able to view and update tickets in their block
- F3.8: Admin shall be able to view and update all tickets system-wide

**F4 — Comment System**

- F4.1: Students shall be able to comment on their own tickets
- F4.2: Wardens and Admins shall be able to comment on any ticket
- F4.3: All comments shall display the author name and timestamp
- F4.4: Comment history shall be visible to all parties on the ticket

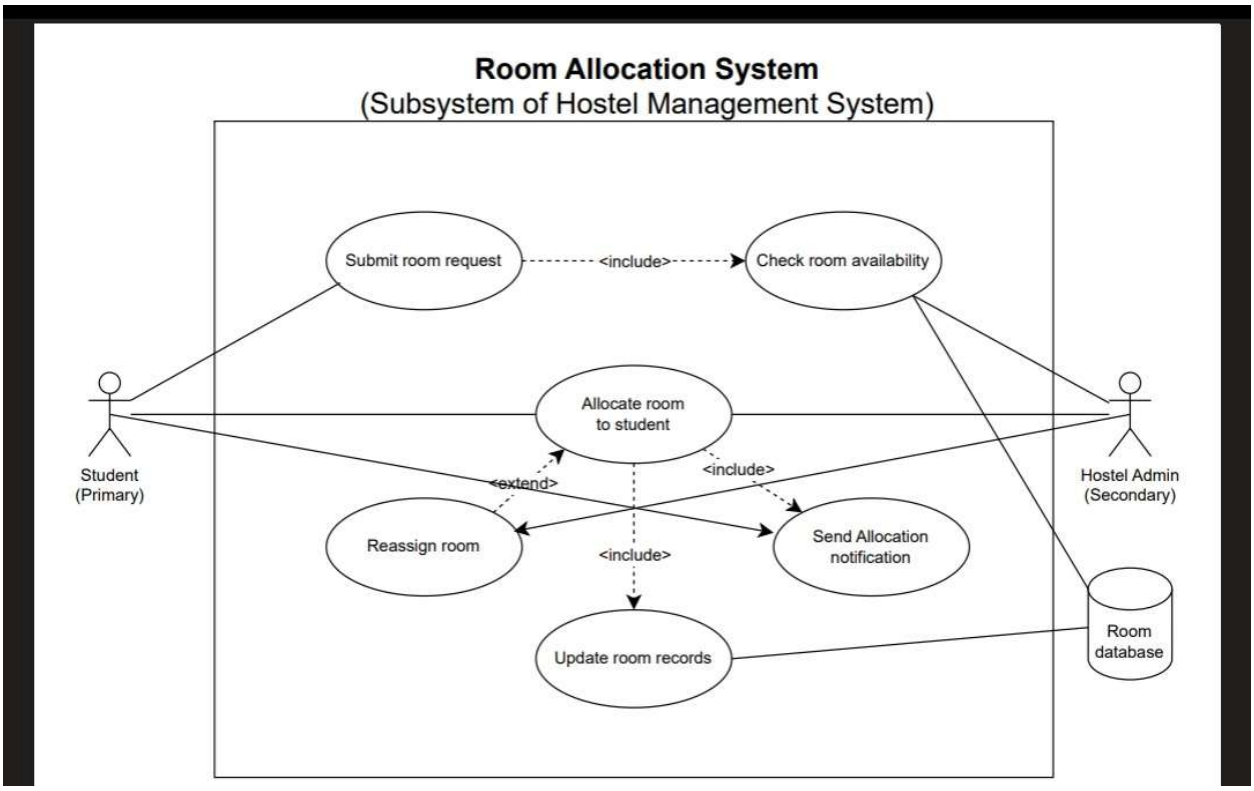
**F5 — Admin Functions**

- F5.1: Admin shall have a dashboard showing system-wide statistics
- F5.2: Admin shall be able to manage user accounts
- F5.3: Admin shall be able to filter tickets by status, category, or date
- F5.4: Admin shall be able to approve or reject room allocation requests

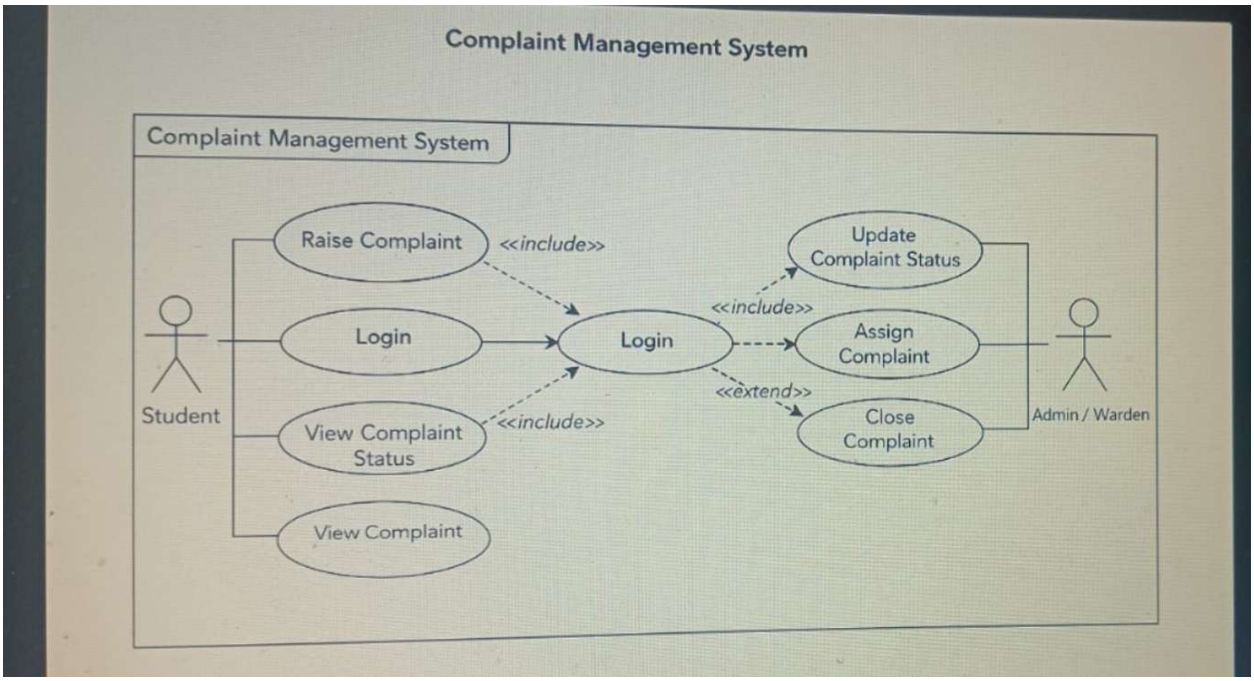
## 3.3 Use Case Model

**Figure 2: Use Case Diagram**

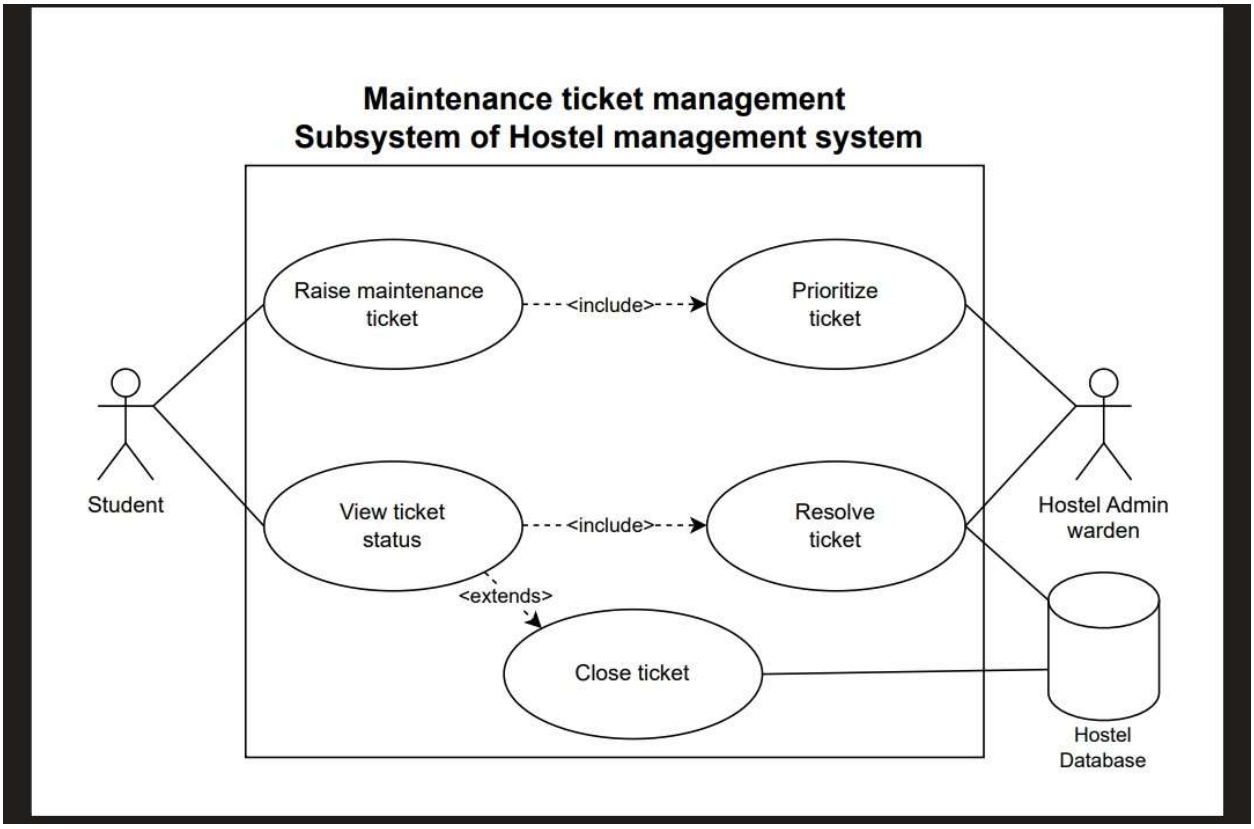
**Use Case Diagram 1 — Room Allocation System (Tathya Sharma | SE23UCSE173)**




**Use Case Diagram 2 — Complaint Management System (NJ Madhav | SE23UCSE122)**




**Use Case Diagram 3 — Maintenance Ticket Management (P Krishna Sai | SE23UMCS064)**



### Use Case U1 — Room Allocation System

| Field | Details |
|---|---|
| Author | Tathya Sharma |
| Purpose | Allow Admin to assign a room to a student |
| Requirements | F2.1, F2.2, F2.3, F2.4 |
| Priority | High |
| Preconditions | Admin is logged in ,room is not at full capacity |
| Post Conditions | Student assigned to room, room status updated to Occupied |
| Actor | Student |
| Basic Flow | Admin selects room → checks availability → assigns student → system saves allocation → notifies student |
| Alternative Flow | Room already occupied → system shows room full error |
| Exceptions | Student already allocated a room → system shows warning |

### Use Case U2 — Complaint Management System

| Field | Details |
|---|---|
| Author | NJ Madhav |
| Purpose | Allow Student to raise a complaint and Admin/Warden to manage it |
| Requirements | F3.1, F3.2, F3.5, F3.7, F3.8 |
| Priority | High |
| Preconditions | Student is logged in |
| Post Conditions | Complaint raised with Pending status, Admin/Warden can assign and update |
| Actor | Student, Admin / Warden |
| Basic Flow | Student logs in → raises complaint → Admin/Warden logs in → assigns complaint → updates status → closes complaint |
| Alternative Flow | Student submits without required fields → validation error shown |
| Exceptions | Database unavailable → error message displayed |

### Use Case U3 — Maintenance Ticket Management

| Field | Details |
|---|---|
| Author | Krishna Sai |

| | |
|---|---|
| Purpose | Allow Student to raise a maintenance ticket and Hostel Admin/Warden to resolve it |
| Requirements | F3.1, F3.2, F3.5, F3.7 |
| Priority | High |
| Preconditions | Student is logged in |
| Post Conditions | Ticket raised, prioritized, and resolved or closed by Admin/Warden |
| Actor | Student, Hostel Admin / Warden |
| Basic Flow | Student raises maintenance ticket → system prioritizes → Admin/Warden views → resolves ticket → ticket closed |
| Alternative Flow | View ticket status → extends to Close ticket if resolved |
| Exceptions | Database error → ticket not saved, error shown |

## 3.4 Other Non-functional Requirements

### 3.4.1 Performance Requirements

- NF1: Page load time shall not exceed 3 seconds under normal conditions
- NF2: The system shall support up to 500 concurrent users
- NF3: API response time for standard requests shall be under 1 second
- NF4: Database shall handle up to 10,000 ticket records without slowdown

### 3.4.2 Safety and Security Requirements

- NF5: Passwords shall be hashed using bcrypt — never stored as plain text
- NF6: All API endpoints shall require a valid JWT token except login
- NF7: Users shall only access data permitted by their role
- NF8: All form inputs shall be validated to prevent SQL injection
- NF9: Sessions shall expire after 24 hours of inactivity

### 3.4.3 Software Quality Attributes

#### 3.4.3.1 Reliability

The system shall be available during hostel operational hours. All database operations shall use transactions to ensure data consistency. Error handling shall be implemented at all API endpoints.

#### 3.4.3.2 Maintainability

The codebase follows a modular structure separating frontend, backend, and database layers. Each module is independently maintainable with consistent naming conventions.

#### 3.4.3.3 Usability

The UI shall be intuitive for first-time users without training. Navigation shall be consistent across all pages. Error messages shall be clear. The system shall work on Chrome, Firefox, and Edge.

#### 3.4.3.4 Adaptability

The system is designed to support future expansion including additional complaint categories, multiple hostel blocks, and email notifications without breaking existing functionality.

## Appendix A — Data Dictionary

| Variable | Type | Description | Related Req. |
|---|---|---|---|
| user_id | INT (PK) | Unique identifier for each user | F1.1 |
| role | ENUM | Student, Warden, or Admin | F1.2 |
| ticket_id | INT (PK) | Unique identifier for each ticket | F3.4 |
| ticket_status | ENUM | Pending, In Progress, Resolved | F3.5 |
| category | ENUM | Electrical, Furniture, Cleanliness | F3.3 |
| room_status | ENUM | Vacant, Partial, Occupied | F2.3 |
| jwt_token | STRING | Auth token issued on login | F1.3 |
| comment_id | INT (PK) | Unique identifier for each comment | F4.1 |
| priority | ENUM | Low, Medium, High | F3.2 |

# Appendix B — Group Log

## Team Information

| # | Name | Roll No | Responsibility |
|---|---|---|---|
| 1 | Tathya Sharma | SE23UCSE173 | Frontend & Backend Development |
| 2 | NJ Madhav | SE23UCSE122 | Database, Integration & Backend |
| 3 | P Krishna Sai | SE23UMCS064 | Frontend & Backend Development |

## Meeting Log

| Date | Attendees | Summary |
|---|---|---|
| 15 Feb 2026 | All 3 | Project kickoff — discussed requirements and divided responsibilities |
| 22 Feb 2026 | All 3 | Finalized tech stack (React, Node.js, MySQL) and SOW |
| 1 Mar 2026 | All 3 | Started UI design — built HTML pages and set up GitHub repository |
| 8 Mar 2026 | All 3 | Completed basic UI — reviewed all pages and prepared Deliverable 1 |

*— End of SRS Document — Team 42 — March 2026 —*